

コードロジック説明書 — BrightStar 統合 LINE アシスタント

版本：v2.0 / 最終更新：2026-09-19

開発者向けに、リポジトリ構成・共通モジュール・各チャネルの処理フローを説明する。

1. リポジトリ構成

```
BrightStar_Line_System/
├── kenshu/      研修 (BS研修)      Python Lambda + Python CDK
│   ├── src/{common, handlers}/    line_webhook.py / webhook.py(WeChat) / web.py / reminder.py
│   └── cdk/                          brightstar_stack.py → stack brightstar-kenshu-dev
├── jinji/      人事 (BS人事)      Python Lambda + TypeScript CDK
│   ├── lambda/{common, handlers}/ line_webhook.py / reconcile.py / business.py / authlib.py
│   ├── templates/                  勤務表.xlsx / 交通費経費.xlsx (S3 hr/template/ へ配備)
│   └── cdk/                          brightstar-hr-stack.ts → stack BrightstarHr-dev
├── soumu/      総務 (BS総務)      Python Lambda + TypeScript CDK
│   ├── lambda/common/              business.py / s3util.py / line.py / assist.py / i18n.py
│   ├── lambda/handlers/            line_webhook.py (回収・督促・DL・削除) / reminder.py (催促・予約・
│   └── cdk/                          brightstar-soumu-stack.ts → stack BrightstarSoumu-dev
├── eigyo/      営業 (BS営業)      Python Lambda + TypeScript CDK
│   ├── transit/   ekitan / geo / cache / batch / models
│   ├── staff/     要員リポジトリ・モデル
│   └── infra/lambda_app/line_handler.py (build.py が build/lambda へ集約)
├── shain/      社員 (BS社員管理)   Python Lambda + Python CDK
│   └── lambda/handler.py             ルーター。kenshu/ と jinji/ を vendoring して同梱
├── docs/       10 システム概要 / 11 機能仕様 / 12 コードロジック / 01-05 利用マニュアル
├── tools/      richmenu.py (Rich Menu 生成・PIL)
└── tests/      pytest (authlib / gate などの自動テスト)
```

CDK の `Code.fromAsset` がリポジトリのソースを直接同梱するため、リポジトリ=デプロイ内容。検証手順と最新の検証結果は [10 システム概要書](#) §8 を参照。

2. 共通モジュール

2-1. `authlib.py` — 認証ゲート

- `gate(channel, uid, text, role_pred) -> (action, item)`：本人確認の中核。花名册照合 → 役割述語 (`role_pred`) で判定し、`pass/ok/tap/need_bind/not_found/ambiguous/wrong_role/taken` を返す。
- `bind_line(empId, uid)`：他の紐付けを解除してから排他登録（占有ロック）。

- `unbind(uid)` : 花名冊の `lineUserId` と認証行を全消去。
- `grant_temp(channel, uid, seconds)` : `validUntil` を書く (テスト用バックドア)。
- `is_authed` : `validUntil` または `authedDate==today(JST)` で当日有効を判定。
- `find_by_line(uid)` / `master_code_today(prefix)` / `today_jst()` 。

2-2. `assist.py` — 多言語・言語設定・ヘルプ

- `resolve(text, intents)` : テキスト完全一致 (大小無視) で canonical intent に解決。多言語同義語を 1 つに畳む。
- `get_lang/set_lang/needs_lang_today(channel, uid)` : 認証(auth)テーブルの `lang` / `langDate` を読み書き。
- `detect_lang_word(text)` : 「日本語／中文」等の言語選択語を検出。
- `quick_reply(text, items)` / `lang_chooser(name)` / `help_message(lang, title, entries)` : Quick Reply メッセージを組み立て。
- `COMMON_INTENTS` : `help` / `auth` / `reset` を共有。

2-3. 共有の配置方法

- `assist.py` / `authlib.py` は各チャネルのビルド時にコピー (eigyo は `infra/build.py` が `build/lambda` へ複製)。
- DynamoDB のテーブル名は **環境変数** (`AUTH_TABLE` / `ROSTER_TABLE` 等) で受け渡し、コードに固定しない。

3. チャネル別フロー

3-1. 共通の受信ループ

```
署名検証(verify_signature) → events をループ
→ 登録解除(RESET_WORDS) ?
→ gate() で本人確認 (action != ok/pass なら案内を返す)
→ 言語選択語 → set_lang → 認証OK+ヘルプ
→ needs_lang_today → 言語チューザー
→ resolve(text, INTENTS) で機能分岐
```

- 返信は `reply_token` (一度きり・約30秒)。Quick Reply 等は `reply_messages` 。

3-2. 研修 (kenshu) `src/handlers/line_webhook.py`

- LINE 前面は **講師(teacher)専用ゲート** (`_teacher_pred`)。受講者は社員アシスタントへ誘導。
- 認証後 `webhook._route(ev)` に委譲し、建課/发布/改課/删課/学员列表/名单/分组 を処理。
- `_handle_event` は str (テキスト) または dict (Quick Reply) を返し、ハンドラが種別で振り分ける。

3-3. 人事 (jinji) `lambda/handlers/line_webhook.py`

- `_route` : バックドア → `reset` → `gate(HR)` → 言語フロー → `resolve(INTENTS)`。
- `INTENTS` : `roster_status` / `missing` / `remind` / `bulk_dl` / `roster_admin` / `proofread` (多言語)。
- `business.roster_of(uid)` は **`roster.lineUserId`** を優先 (`identity` の単一情報源)。
- 催促は別 Lambda (`REMINDER_FUNCTION_NAME`) を非同期 `invoke`。提出済は差集合で除外。

3-4. 営業 (eigyo) `infra/lambda_app/line_handler.py`

処理順 (認証後) : 1. 言語選択 → 認証OK+ヘルプ 2. `needs_lang_today` → 言語チューザー 3. `help` 4. CSV ファイル受信 → `_import_staff_csv` (列ずれ検知 → 楽観ロック → 住所→駅変換 → `bulk_upsert`) 5. 要員一覧DL → `_export_to_s3` (**`QUOTE_ALL`** で全項目引用) + 署名付き DL リンク 6. `|` を含む → 単票 CRUD (`_handle_crud`) 7. **明示プレフィックス** (`_extract_site`) のときだけ現場比較。無ければ案内のみ 8. 比較は `ack(reply)` → 自己 `invoke` の `_worker` で重い取得を実行し **push** (結果+CSVダウンロードボタン)

通勤比較の中核 : - `batch.compare_site(site, staffs, source, registry)` : 各要員の最寄駅→現場を取得。同一 (`from_id, to_id`) は実行内メモでキャッシュ。 `StaffCommute` に `from_id/to_id` を保持し確認URLを生成。 - `ekitan_source.EkitanScraper` : 運賃ページ+定期ページを取得し路線シグネチャで突合。 `_route_summary` は「駅→[路線名]→駅」を組み立て、徒歩連絡を〔徒歩〕で明示 (乗換回数と駅数の差を説明可能に)。 - `transit/geo.py nearest_station(住所)` : Amazon Location `geo-places` で住所→最寄駅。 - `DynamoDBCache` : 経路結果をキャッシュ (`cacheKey = namespace#from_to`、TTLあり)。

3-5. 社員 (shain) `lambda/handler.py`

- `from jinji.common import assist/authlib/business`、`from kenshu... import ...` (名前空間分離 **vendoring**)。
- フロー : `reset` → 初回登録ゲート (`jbiz.handle_registration`) → 言語フロー → ヘルプ → モード切替 → ファイルは人事固定 → 当日モードで研修/人事へ委譲。
- 研修委譲時は `_ensure_kenshu_student(uid, name)` で受講者を自動プロビジョニング (二重登録防止)。
- セッション (mode + 日付) は専用テーブル (`SHAIN_SESSION_TABLE`) に保持。

4. vendoring (名前空間分離) の要点

- `kenshu` と `jinji` は `common` / `handlers` のモジュール名が衝突する。社員 Lambda では両者を `kenshu.*` / `jinji.*` のサブパッケージに収め、**絶対 import を sed で書換え** (`from common` → `from kenshu.common`)、相対 import は据え置く。
- これにより 2 つの Python バックエンドを 1 Lambda に同梱しても衝突しない。

5. データモデル (DynamoDB)

テーブル	キー	主な属性
花名册 roster	empId (PK)	name / department / role / lineUserId
認証 auth	pk = channel#userId	authedDate / lang / langDate / validUntil / expireAt(TTL)
courses (研修)	courseId	title / startTime / durationMin / capacity / status / zoom* / remind1hSent
enrollments (研修)	(openid, courseId)	status / enrolledAt
submissions (提出)	userId + sk= period#type (その他経費は period#other#ts)	s3Key / fileName / submittedAt。社員が登録、総務が参照・削除
employees (紐付け)	userId	社員番号 / status / expectType・expectRaw (次ファイルの種別事前指定)
session (社員)	userId	当日のモード (研修 / 総務)
bookings (総務)	bookingId	runAtEpoch / runAtJst / status / createdBy (催促予約)
broadcasts (総務)	bcastId	kind / createdAt / targets[] / confirmed{userId→{at,name}} (既読確認)
StaffTable (営業)	staff_id	name / nearest_station / department / status / updated_at
CacheTable (営業)	cacheKey	payload(JSON) / cachedAt / ttl

- 営業の updated_at は **マイクロ秒精度** (同一秒内更新でも楽観ロックが機能するため)。

6. テスト

- tests/test_authlib.py : gate の各状態、一括認証、占有ロック、unbind の排他、temp grant / master code を pytest + moto で検証。
- 注意: moto は **geo-places を未サポート**。住所→駅変換の結合は実 Lambda で確認する (CSV/単票で検証済み)。

7. ブロック/削除の検知 (2026-06 追加)

7-1. リアルタイム (webhook)

- `authlib.mark_blocked(user_id, blocked)` / `list_blocked()` : roster の `blocked` / `blockedAt` を更新 (lineUserId 一致のエントリ。予約語回避で `ExpressionAttributeNames` 使用)。
- 各チャンネルの webhook で `unfollow` → `mark_blocked(True)` + `[UNFOLLOW]` ログ、`follow` → `mark_blocked(False)`。kenshu の `line.py parse_events` に `unfollow→unsubscribe` を追加。
- 人事 `_roster_list_msg` がブロック中を  ブロック中 で表示。

7-2. 日次リコンサイル (`handlers/reconcile.py`)

- jinji スタックの **ReconcileFunction** + `EventBridge cron(0 15 * * ? *)` (=0:00 JST) で起動。
- `_relevant_channels(item)` : 役割/部署で「使えるチャンネル」を決める (社員=全員 / teacher→研修 / hr→人事 / sales・営業部→営業)。
- `_reachable(uid, channels)` : 対象チャンネルの `GET /v2/bot/profile/{userId}` を確認。全て 401/403/404 のときのみブロック/削除と判定 (429/5xx・一時障害は安全側に到達可能)。
- 判定されたら `authlib.unbind()` で **lineUserId と全チャンネルの認証行をクリア**。4 チャンネルの token は SSM から読む (IAM で 4 パラメータの `GetParameter` を許可)。

8. 主要テーブルの役割 (補足)

テーブル	役割	書き込む箇所
roster (花名冊)	本人情報の正本。lineUserId が紐付けの真実	人事の名簿管理、各チャンネルの認証 (bind)
auth	日次認証状態 + 言語 (pk= channel#userId、TTL 2日)	各チャンネルの gate / 言語選択
employees	社員チャンネル専用 の登録台帳 + 提出の一時ポインタ (<code>pendingKey</code>)	社員の登録 (<code>handle_registration</code>) ・提出フロー
brightstar-shain-dev-session	社員の当日モード (研修/人事) 保持	社員チャンネル

「誰が紐付け済みか / 今日認証したか」を見る正本は **roster.lineUserId + auth**。
employees は社員チャンネルの登録 + 提出一時状態に使う (他チャンネルは書かない)。

追補 (2026-07) : 提出・催促の拡張

- **形式判定** `shain/lambda/jinji/common/s3util.py: detect_format(data)` … 先頭バイトで `xlsx(PK)/PDF(%PDF)/JPEG/PNG` を判定。 `put_submission(ext, mime)` が実形式で保存、`content_type_for(key)` が /dl の Content-Type を返す。
- **提出分岐** `shain/.../handlers/line_webhook.py: _handle_file` … `expectType` (提出ボタンで事前確定) → その他経費モード → ファイル名推定の順に種別を決め、xlsx は `_do_submit` (全チェック)、PDF/画像は `_do_submit_raw` (保存のみ + 目視確認案内)。

- **その他経費** … employees.otherStep/otherPurpose の 2 ステップ状態機。 save_other_submission が S3 others/ と submissions (SK= {period}#other#{ts}) へ。
- **手動「済」** soumu/lambda/common/business.py: manual_ok_set/clear … roster.manualOk (監査つき)。 _submitted(person, done, period, type) が manualOk を提出扱いに。表示は _emp_line の ✓(手)。
- **催促予約** soumu … line.py (postback 解析 + datetime_picker_message) → webhook (催促予約/予約確認/予約取消、postback book_remind) → business (booking_add/bookings_due/booking_mark/booking_cancel、JST→epoch) → reminder.py (trigger:"poll" で期限到来分を _send_reminders 実行)。CDK: BookingsTable + rate(10 minutes) の BookingPoller。
- **リッチメニュー** tools/richmenu.py … build() をセル全面カード + 自動フォントサイズに刷新 (絵文字廃止)。soumu メニュー新設、全5チャンネル再生成。